МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Дисциплина «Структуры и алгоритмы обработки данных»

Отчет по лабораторной работе №4

Выполнила:

Студент группы БВТ2504
Мясникова Е.А.

Проверила: Кудрявцев В.Д.

Москва, 2026 г.

Содержание

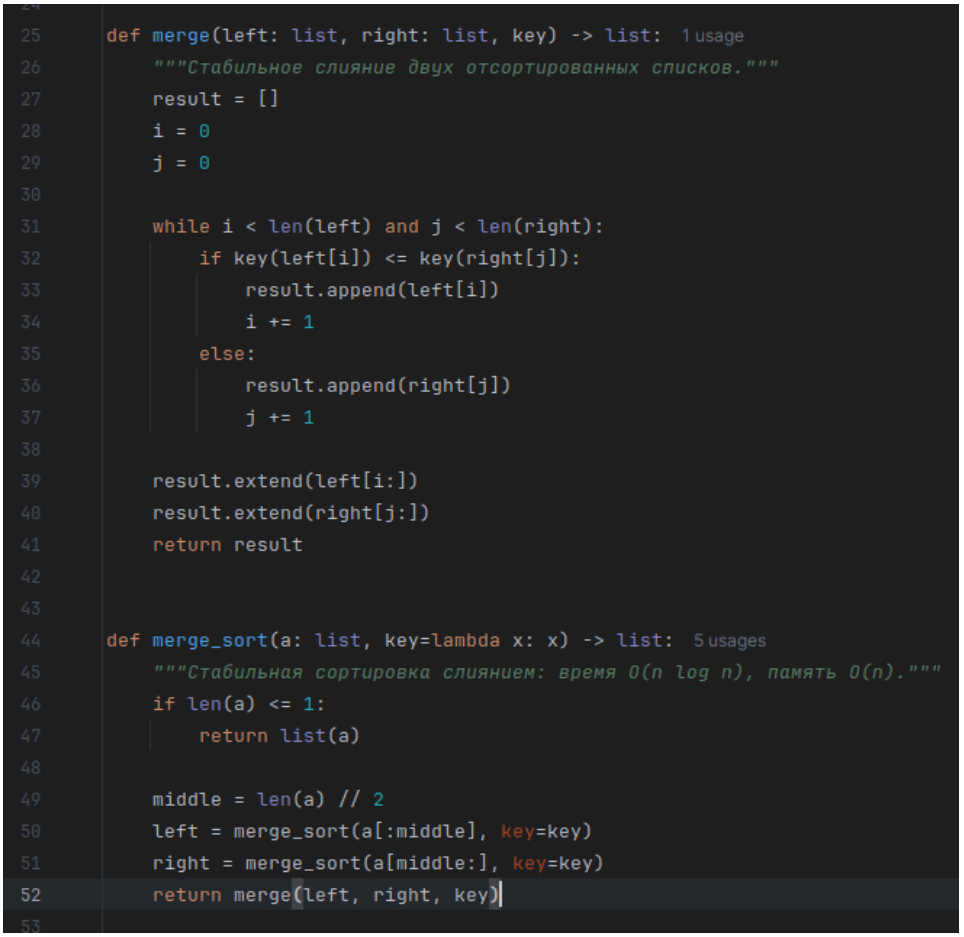
Цель работы:.....2
Ход выполненной работы:.....2
1. Задание 1.....2
2. Задание 2.....3
3. Задание 3.....5
4. Задание 4.....6
5. Задание 5.....7
6. Задание 6.....15
7. Задание 7.....16
Заключение.....19

Цель работы: Освоить сортировки с гарантией O(n log n) и линейно-временные сортировки; научиться сопоставлять их свойства (время, память, стабильность, ограничения применимости) и выбирать алгоритм по характеру данных.

Ход выполненной работы:

1. **Задание 1**

Реализовали сортировку слиянием MergeSort. Она делит список на половины, пока не останутся части из одного элемента, а затем соединяет их в порядке возрастания. При одинаковых значениях первым берётся элемент из левой половины, поэтому исходный порядок таких элементов сохраняется. Правильность работы проверили сравнением со встроенной сортировкой sorted().



```
25  def merge(left: list, right: list, key) -> list:  1 usage
26      """Стабильное слияние двух отсортированных списков."""
27      result = []
28      i = 0
29      j = 0
30
31      while i < len(left) and j < len(right):
32          if key(left[i]) <= key(right[j]):
33              result.append(left[i])
34              i += 1
35          else:
36              result.append(right[j])
37              j += 1
38
39      result.extend(left[i:])
40      result.extend(right[j:])
41      return result
42
43
44  def merge_sort(a: list, key=lambda x: x) -> list:  5 usages
45      """Стабильная сортировка слиянием: время O(n log n), память O(n)."""
46      if len(a) <= 1:
47          return list(a)
48
49      middle = len(a) // 2
50      left = merge_sort(a[:middle], key=key)
51      right = merge_sort(a[middle:], key=key)
52      return merge(left, right, key)
53
```

Рисунок 1 - Код реализации сортировки слиянием MergeSort

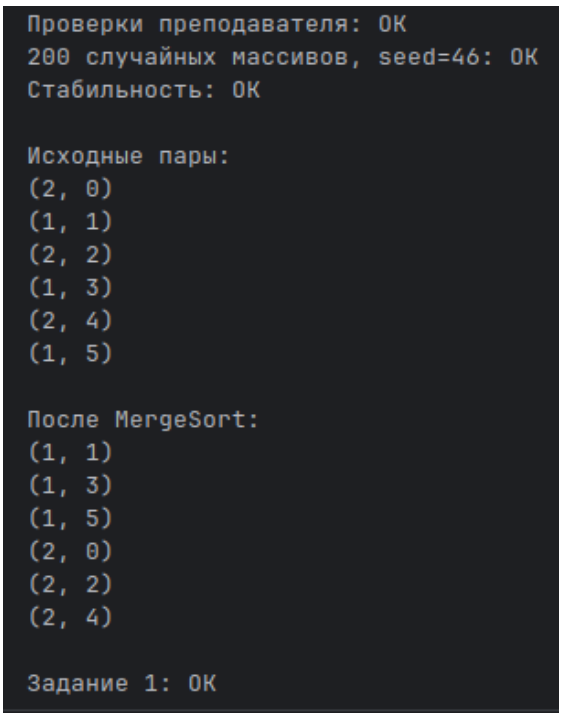


Рисунок 2 - Результаты проверки сортировки MergeSort

Вывод: Все проверки прошли успешно: результаты MergeSort совпали с sorted(). Проверка на парах подтвердила, что элементы с одинаковым значением для сортировки сохранили исходный порядок.

2. Задание 2

Реализована сортировка подсчётом CountingSort. Сначала программа подсчитывает количество элементов с каждым ключом. Затем вычисляет накопленные количества - префиксные суммы, которые помогают определить места элементов в результате. Исходный список обходится с конца, чтобы элементы с одинаковым ключом сохранили свой порядок. Пусть n - количество элементов, а k = max – min - разница между наибольшим и наименьшим ключом. В этой реализации массив счётчиков содержит k + 1 ячейку, поэтому время работы и дополнительная память оцениваются как O(n + k). CountingSort подходит для небольшого диапазона ключей. Если значений мало, но они сильно отличаются друг от друга, приходится создавать большой массив счётчиков, и алгоритм становится невыгодным.

```python
def counting_sort(a: list, key_max: int, key=lambda x: x) -> list:  1 usage
    """Устойчивая сортировка целых ключей от 0 до key_max."""
    if not a:
        return []

    minimum = min(key(item) for item in a)
    maximum = max(key(item) for item in a)
    if minimum < 0 or maximum > key_max:
        raise ValueError("Ключ выходит за диапазон от 0 до key_max")

    counts = [0] * (maximum - minimum + 1)
    for item in a:
        counts[key(item) - minimum] += 1

    for i in range(1, len(counts)):
        counts[i] += counts[i - 1]

    result = [None] * len(a)
    for item in reversed(a):
        index = key(item) - minimum
        counts[index] -= 1
        result[counts[index]] = item

    return result
```

Рисунок 3 - Реализация сортировки подсчётом CountingSort

```
Проверки заготовки: OK
200 случайных массивов, seed=46: OK
Устойчивость: OK

Исходные пары:
(2, 0)
(1, 1)
(2, 2)
(1, 3)
(2, 4)
(1, 5)

После CountingSort:
(1, 1)
(1, 3)
(1, 5)
(2, 0)
(2, 2)
(2, 4)

Задание 2: OK
```

Рисунок 4 - Проверка правильности и устойчивости CountingSort

Вывод: CountingSort сохраняет порядок элементов с одинаковым ключом. Алгоритм подходит для небольшого диапазона ключей: при большом значении k = max - min увеличиваются затраты времени и памяти.

3. Задание 3

Реализована поразрядная сортировка RadixSort LSD для неотрицательных целых ключей. Числа последовательно сортируются по единицам, десяткам и следующим разрядам с помощью устойчивой сортировки подсчётом. Устойчивость каждого прохода сохраняет порядок, полученный при обработке младших разрядов.

```python
def counting_sort_digit(a: list, exp: int, key=lambda x: x) -> list:  1 usage  new *
    """Устойчивый проход по одной десятичной цифре."""
    return counting_sort(a, key_max=9, key=lambda item: (key(item) // exp) % 10)


def radix_sort_lsd(a: list, key=lambda x: x) -> list:  2 usages  new *
    """RadixSort LSD: время O(d(n + 10)), память O(n + 10)."""
    if not a:
        return []
    if min(key(item) for item in a) < 0:
        raise ValueError("RadixSort принимает неотрицательные целые ключи")

    result = list(a)
    maximum = max(key(item) for item in result)
    exp = 1
    while exp <= maximum:
        result = counting_sort_digit(result, exp, key=key)
        exp *= 10
    return result
```

Рисунок 5 - Реализация поразрядной сортировки RadixSort LSD

```
Проверки заготовки: OK
200 случайных массивов, seed=46: OK
Устойчивость RadixSort и прохода: OK

Исходные числа:
21
12
22
11

После прохода по единицам:
21
11
12
22

После прохода по десяткам:
11
12
21
22

Задание 3: OK
```

Рисунок 6 - Проверка правильности и устойчивости RadixSort LSD

Вывод: Результаты RadixSort совпали со встроенной сортировкой sorted(). Проверка подтвердила устойчивость поразрядных проходов и всей сортировки.

4. Задание 4

Реализована пирамидальная сортировка HeapSort. Сначала из списка строится max-куча, в которой наибольший элемент находится в начале. Затем максимум последовательно переносится в конец рабочей части, а свойство кучи восстанавливается просейкой вниз. Сортировка выполняется в исходном списке без вспомогательного массива. Построение кучи занимает O(n), сортировка в худшем случае - O(n log n), дополнительная память - O(1).

```python
def sift_down(a: list[int], i: int, n: int) -> None:  2 usages  Lena *
    """Просейка вниз в max-куче: рассматриваются индексы меньше n."""
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2

        if left < n and a[left] > a[largest]:
            largest = left
        if right < n and a[right] > a[largest]:
            largest = right
        if largest == i:
            return

        a[i], a[largest] = a[largest], a[i]
        i = largest


def build_heap(a: list[int]) -> None:  1 usage  new *
    """Построение max-кучи снизу вверх за O(n)."""
    for i in range(len(a) // 2 - 1, -1, -1):
        sift_down(a, i, len(a))


def heap_sort(a: list[int]) -> None:  2 usages  Lena *
    """Сортировка на месте: худшее время O(n log n), память O(1)."""
    build_heap(a)
    for end in range(len(a) - 1, 0, -1):
        a[0], a[end] = a[end], a[0]
        sift_down(a,  i: 0, end)
```

Рисунок 7 - Реализация пирамидальной сортировки HeapSort

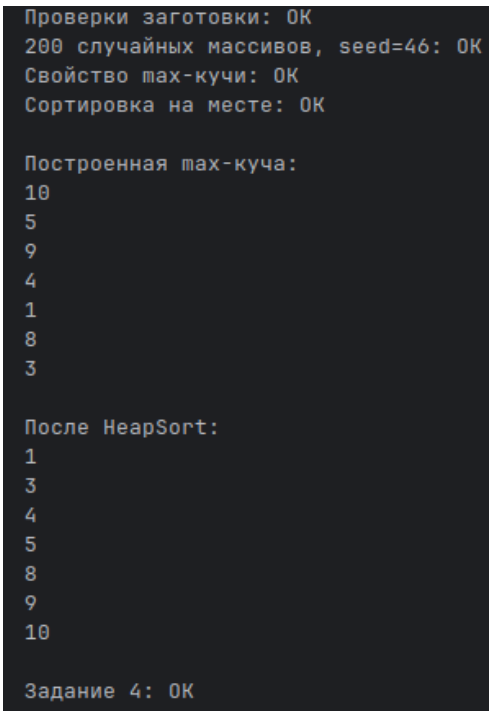
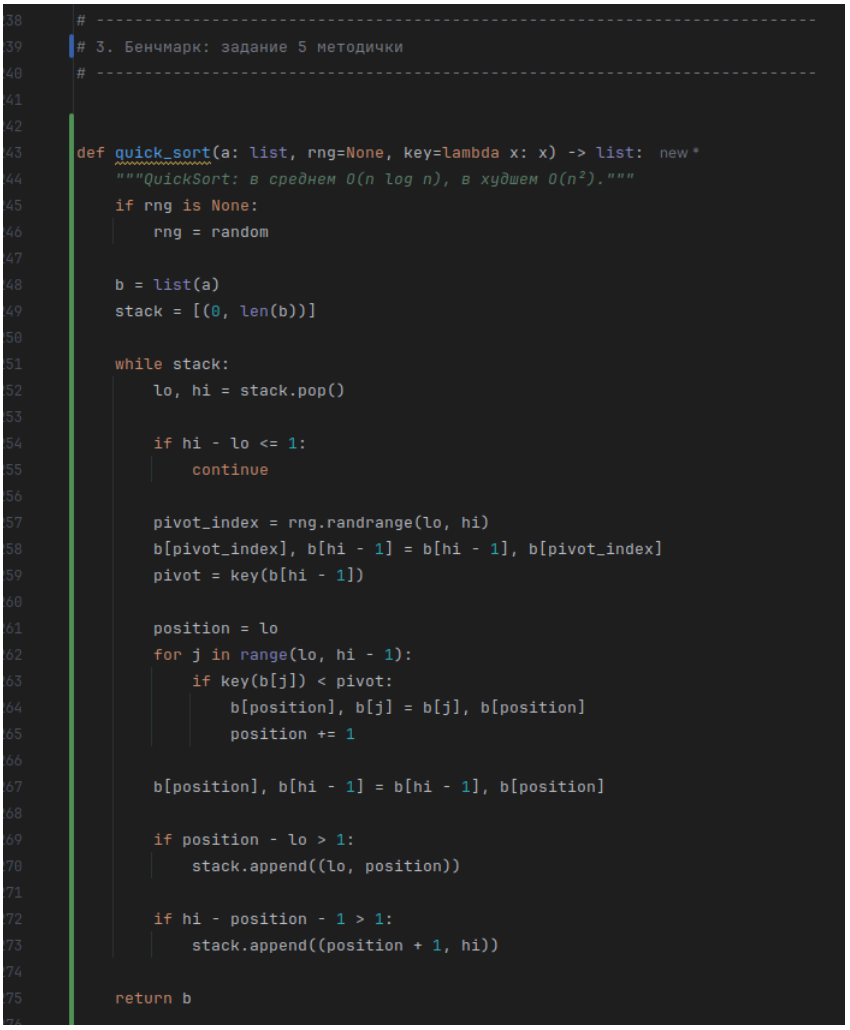


Рисунок 8 - Проверка построения кучи и работы HeapSort

Вывод: HeapSort правильно отсортировал проверенные списки. Дополнительно подтверждены свойство max-кучи после построения и сортировка переданного списка на месте.

5. Задание 5

Сравнили MergeSort, CountingSort, RadixSort, HeapSort, QuickSort из ЛР3 и встроенную функцию sorted(). Использовали пять размеров массивов от 1000 до 100000 элементов и пять распределений данных. Для каждой точки выполнили один прогрев и пять измеряемых запусков, затем определили медиану времени. Подготовку данных и создание копии массива выполняли до включения таймера. Результаты сохранили в CSV и построили графики.



```python
# ---------------------------------------------------------------------------
# 3. Бенчмарк: задание 5 методички
# ---------------------------------------------------------------------------


def quick_sort(a: list, rng=None, key=lambda x: x) -> list:  new *
    """QuickSort: в среднем O(n log n), в худшем O(n²)."""
    if rng is None:
        rng = random

    b = list(a)
    stack = [(0, len(b))]

    while stack:
        lo, hi = stack.pop()

        if hi - lo <= 1:
            continue

        pivot_index = rng.randrange(lo, hi)
        b[pivot_index], b[hi - 1] = b[hi - 1], b[pivot_index]
        pivot = key(b[hi - 1])

        position = lo
        for j in range(lo, hi - 1):
            if key(b[j]) < pivot:
                b[position], b[j] = b[j], b[position]
                position += 1

        b[position], b[hi - 1] = b[hi - 1], b[position]

        if position - lo > 1:
            stack.append((lo, position))

        if hi - position - 1 > 1:
            stack.append((position + 1, hi))

    return b
```

Рисунок 9 - QuickSort из лабораторной работы №3

```python
SIZES = [1_000, 3_000, 10_000, 30_000, 100_000]
REPEATS = 5
SMALL_KEY_MAX = 100
WIDE_KEY_MAX = 10**9


def make_datasets(rng: random.Random, n: int) -> dict[str, list[int]]:  1 usage  Lena *
    """Пять распределений из заготовки преподавателя."""
    almost = list(range(n))
    for _ in range(max(1, n // 100)):
        i, j = rng.randrange(n), rng.randrange(n)
        almost[i], almost[j] = almost[j], almost[i]
    return {
        "random_wide": [rng.randint( a: 0, WIDE_KEY_MAX) for _ in range(n)],
        "random_small": [rng.randint( a: 0, SMALL_KEY_MAX) for _ in range(n)],
        "sorted": list(range(n)),
        "reversed": list(range(n, 0, -1)),
        "almost_sorted": almost,
    }


def bench(fn, data: list[int]) -> float:  Lena *
    """Один прогрев, пять измеряемых повторов и медиана."""
    fn(list(data))
    times = []
    for _ in range(REPEATS):
        buf = list(data)
        start = time.perf_counter()
        fn(buf)
        times.append(time.perf_counter() - start)
    return statistics.median(times)
```

Рисунок 10 - Подготовка массивов и измерение времени сортировки

```python
def plot_results(rows: list, folder, seed: int) -> None:  1 usage  new *
    """Пять log-log графиков по результатам, сохранённым в CSV."""
    import matplotlib

    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    for distribution in dict.fromkeys(row[1] for row in rows):
        fig, ax = plt.subplots(figsize=(8, 5))
        for name in dict.fromkeys(row[2] for row in rows):
            points = [
                row for row in rows
                if row[1] == distribution and row[2] == name
                and row[3] is not None
            ]
            if points:
                ax.loglog(
                    *args: [row[0] for row in points], [row[3] for row in points],
                    marker="o", label=name,
                )
        ax.set_title(f"{distribution}; seed={seed}")
        ax.set_xlabel("Количество элементов n")
        ax.set_ylabel("Медиана времени, с")
        ax.grid( visible: True, which="both", alpha=0.3)
        ax.legend()
        fig.tight_layout()
        path = folder / f"{distribution}_time.png"
        fig.savefig(path, dpi=160)
        plt.close(fig)
        print("Сохранён график:", path.name)
```

Рисунок 11 - Построение графиков по результатам измерений

```python
def run_benchmarks(seed: int) -> None:  Lena *
    """Все сортировки получают одинаковые входные данные."""
    import csv
    from pathlib import Path

    rng = random.Random(seed)
    pivot_rng = random.Random(seed)
    algorithms = [
        ("merge_sort", merge_sort),
        ("counting_sort", lambda a: counting_sort(a, key_max=max(a, default=0))),
        ("radix_sort_lsd", radix_sort_lsd),
        ("heap_sort", heap_sort),
        ("quick_sort", lambda a: quick_sort(a, rng=pivot_rng)),
        ("sorted", sorted),
    ]
    rows = []
    for n in SIZES:
        for distribution, data in make_datasets(rng, n).items():
            print(f"\nn={n}; {distribution}", flush=True)
            for name, fn in algorithms:
                seconds = None
                if name == "counting_sort" and distribution == "random_wide":
                    print(f"  {name}: пропуск — широкий диапазон", flush=True)
                else:
                    seconds = bench(fn, data)
                    print(f"  {name}: {seconds:.8f} c", flush=True)
                rows.append((n, distribution, name, seconds))
    folder = Path(__file__).resolve().parent
    with (folder / "results.csv").open( mode: "w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["n", "distribution", "algorithm", "seconds"])
        writer.writerows(rows)
    print(f"\nСохранено {len(rows)} строк в results.csv")
    plot_results(rows, folder, seed)
```

Рисунок 12 - Сравнение шести сортировок и сохранение результатов

```python
def main() -> None:  Lena *
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument( *name_or_flags: "--variant", type=int, required=True, help="номер варианта")
    args = ap.parse_args()
    seed = 30 + args.variant
    random.seed(seed)
    print(f"Вариант: {args.variant}; seed={seed}")
    print("Python:", sys.version.split()[0])
    print(f"Методика: 1 прогрев, {REPEATS} повторов, медиана")
    self_check()
    run_benchmarks(seed)


if __name__ == "__main__":
    main()
```

Рисунок 13 - Запуск проверок и измерений для своего варианта

```python
def quick_sort(a: list, rng=None, key=lambda x: x) -> list:  new *
    """QuickSort: в среднем O(n log n), в худшем O(n²)."""
    if rng is None:
        rng = random

    b = list(a)
    stack = [(0, len(b))]

    while stack:
        lo, hi = stack.pop()

        if hi - lo <= 1:
            continue

        pivot_index = rng.randrange(lo, hi)
        b[pivot_index], b[hi - 1] = b[hi - 1], b[pivot_index]
        pivot = key(b[hi - 1])

        position = lo
        for j in range(lo, hi - 1):
            if key(b[j]) < pivot:
                b[position], b[j] = b[j], b[position]
                position += 1

        b[position], b[hi - 1] = b[hi - 1], b[position]

        if position - lo > 1:
            stack.append((lo, position))

        if hi - position - 1 > 1:
            stack.append((position + 1, hi))

    return b
```

Рисунок 14 - Реализация QuickSort из лабораторной работы №3

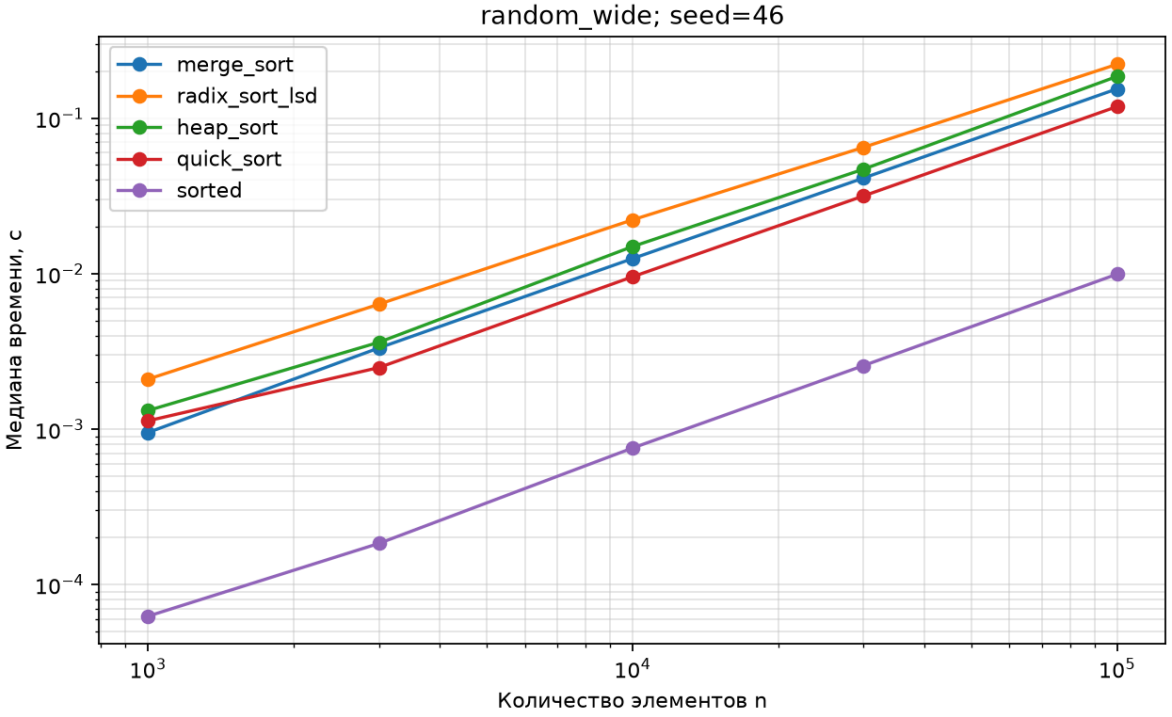


Рисунок 15 - Время сортировки случайных массивов с широким диапазоном ключей

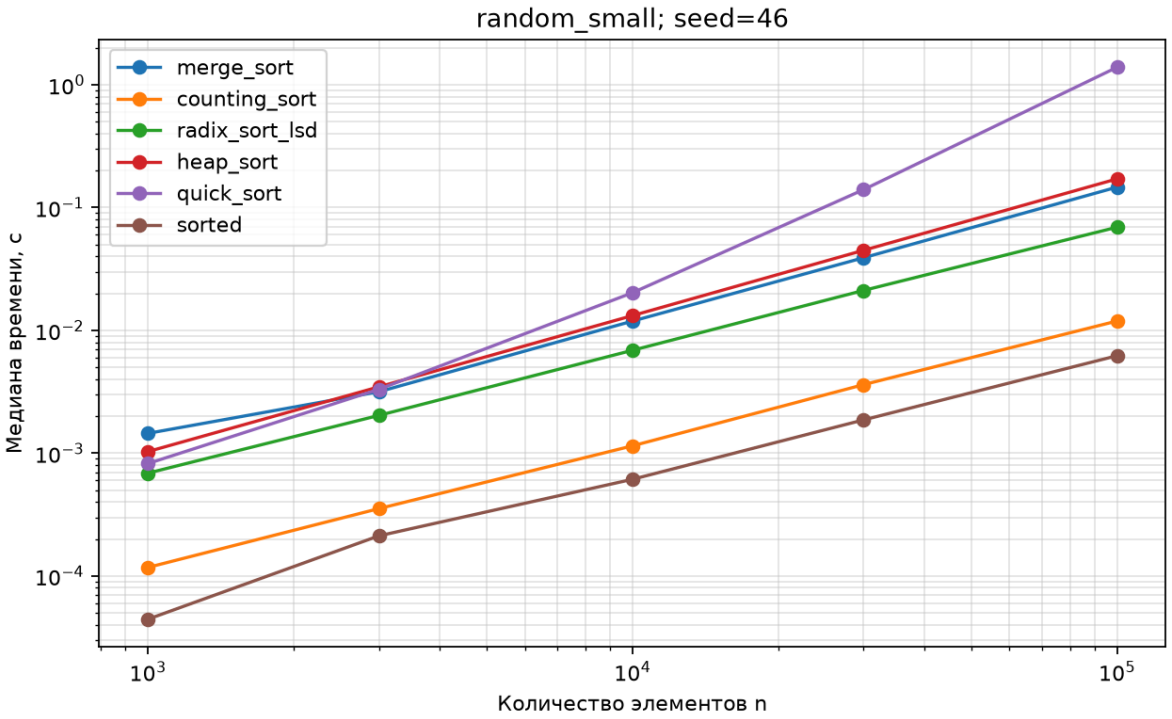


Рисунок 16 - Время сортировки случайных массивов с узким диапазоном ключей

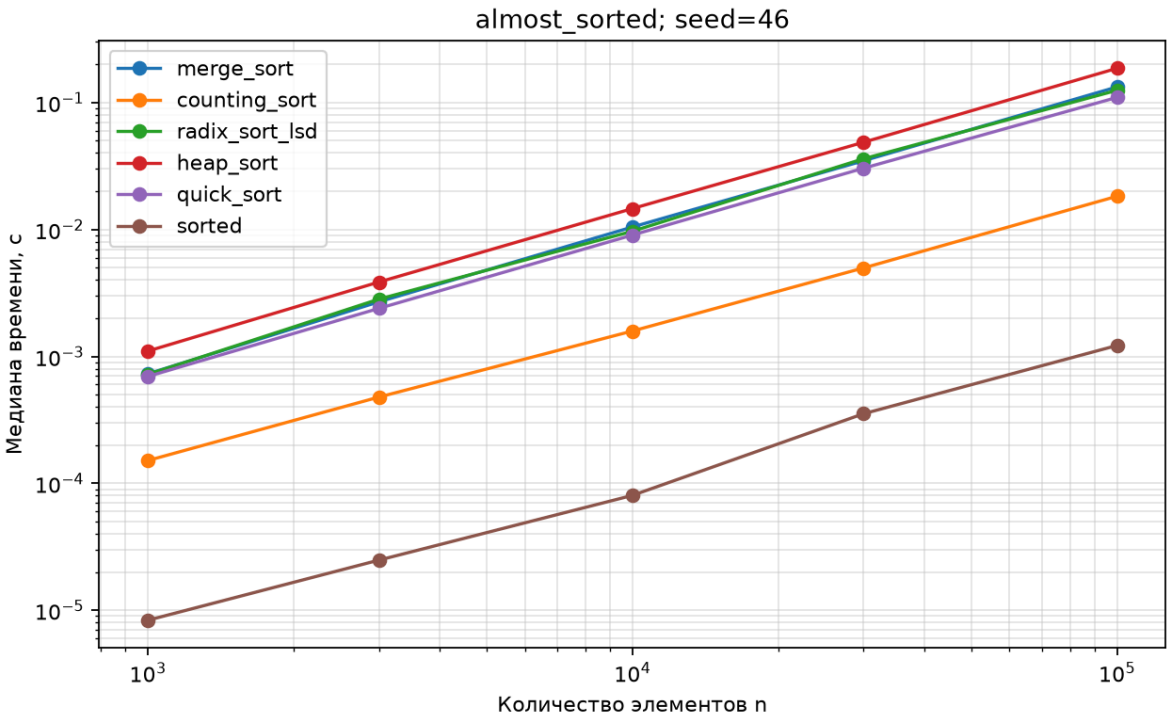


Рисунок 17 - Время сортировки отсортированных массивов

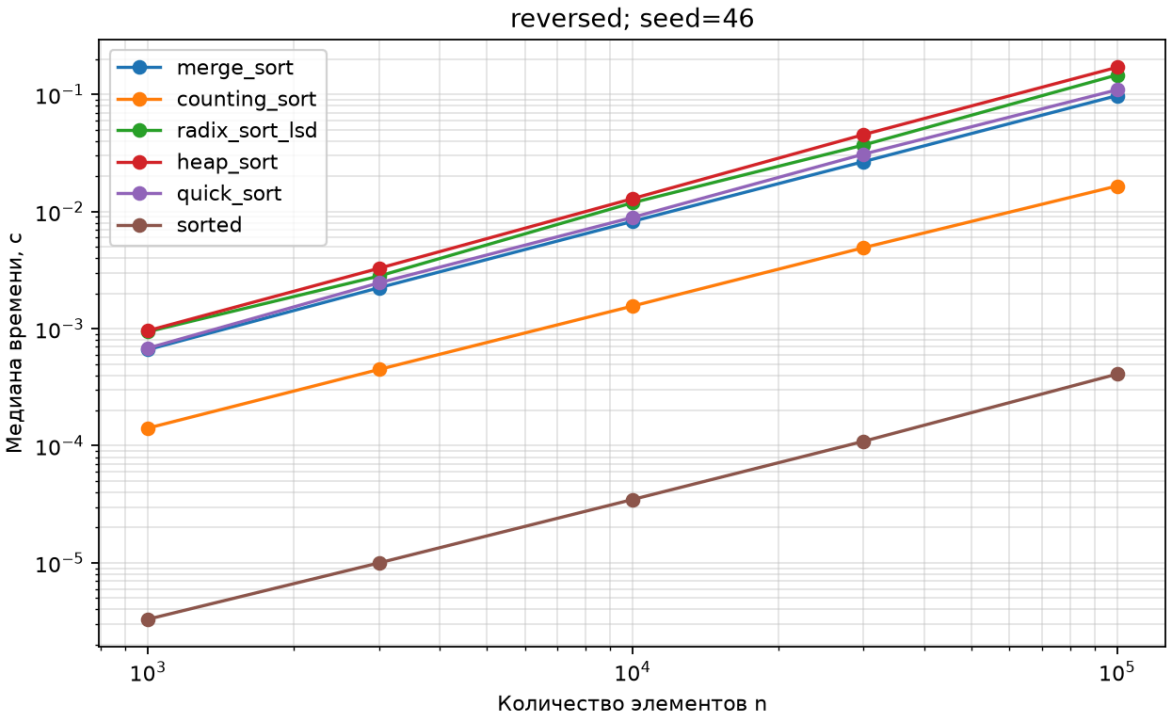


Рисунок 18 - Время сортировки массивов в обратном порядке

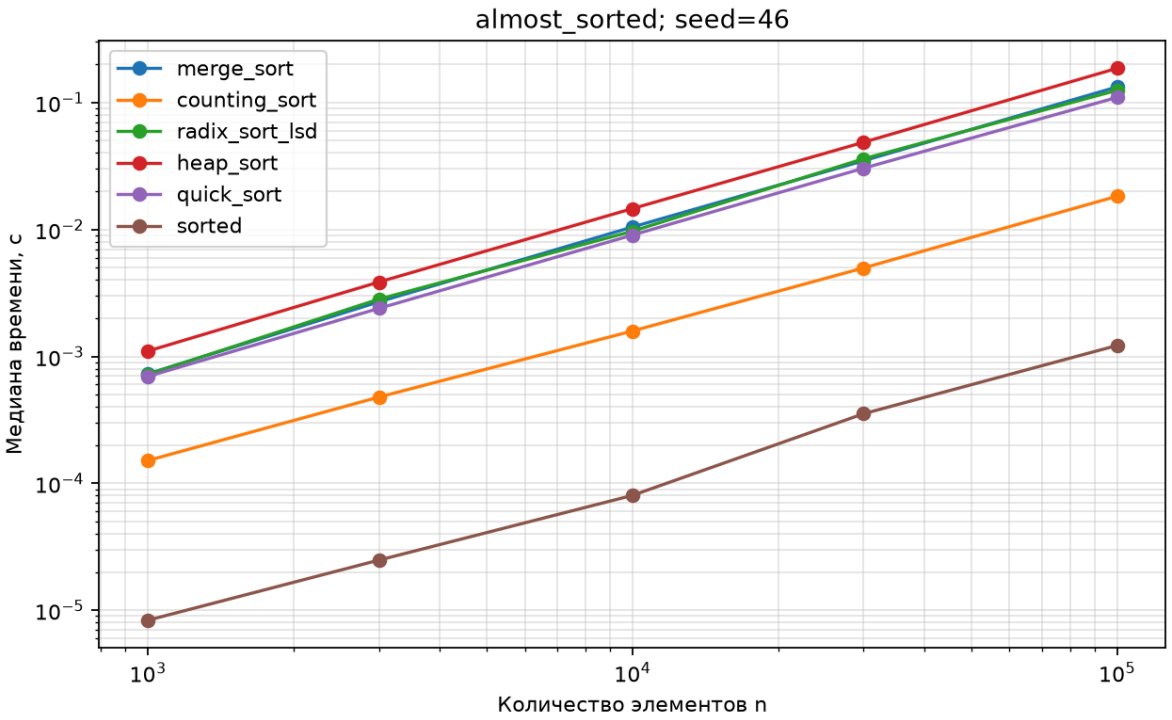
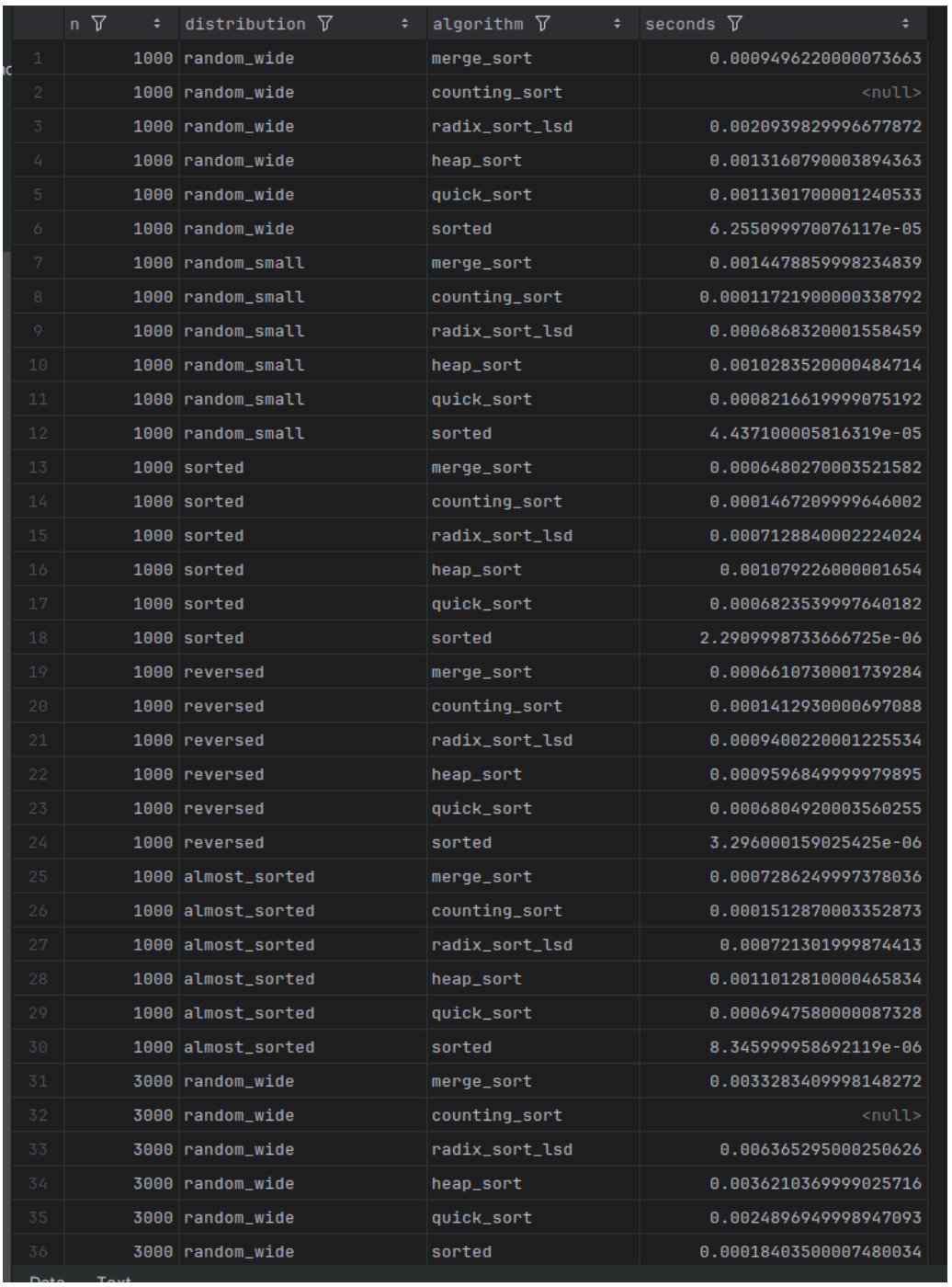


Рисунок 19 - Время сортировки почти отсортированных массивов



	n	distribution	algorithm	seconds
1	1000	random_wide	merge_sort	0.0009496220000073663
2	1000	random_wide	counting_sort	<null>
3	1000	random_wide	radix_sort_lsd	0.0020939829996677872
4	1000	random_wide	heap_sort	0.0013160790003894363
5	1000	random_wide	quick_sort	0.0011301700001240533
6	1000	random_wide	sorted	6.255099970076117e-05
7	1000	random_small	merge_sort	0.0014478859998234839
8	1000	random_small	counting_sort	0.0001171900000338792
9	1000	random_small	radix_sort_lsd	0.0006868320001558459
10	1000	random_small	heap_sort	0.0010283520000484714
11	1000	random_small	quick_sort	0.0008216619999075192
12	1000	random_small	sorted	4.437100005816319e-05
13	1000	sorted	merge_sort	0.0006480270003521582
14	1000	sorted	counting_sort	0.0001467209999646002
15	1000	sorted	radix_sort_lsd	0.0007128840002224024
16	1000	sorted	heap_sort	0.001079226000001654
17	1000	sorted	quick_sort	0.0006823539997640182
18	1000	sorted	sorted	2.2909998733666725e-06
19	1000	reversed	merge_sort	0.0006610730001739284
20	1000	reversed	counting_sort	0.0001412930000697088
21	1000	reversed	radix_sort_lsd	0.0009400220001225534
22	1000	reversed	heap_sort	0.0009596849999979895
23	1000	reversed	quick_sort	0.0006804920003560255
24	1000	reversed	sorted	3.296000159025425e-06
25	1000	almost_sorted	merge_sort	0.0007286249997378036
26	1000	almost_sorted	counting_sort	0.0001512870003352873
27	1000	almost_sorted	radix_sort_lsd	0.000721301999874413
28	1000	almost_sorted	heap_sort	0.0011012810000465834
29	1000	almost_sorted	quick_sort	0.0006947580000087328
30	1000	almost_sorted	sorted	8.345999958692119e-06
31	3000	random_wide	merge_sort	0.0033283409998148272
32	3000	random_wide	counting_sort	<null>
33	3000	random_wide	radix_sort_lsd	0.006365295000250626
34	3000	random_wide	heap_sort	0.0036210369999025716
35	3000	random_wide	quick_sort	0.0024896949998947093
36	3000	random_wide	sorted	0.0001840350000748003

Рисунок 20 - Фрагмент таблицы результатов измерения времени сортировок

Вывод: В проведённых измерениях sorted() оказался самым быстрым на всех наборах. Среди собственных реализаций на узком диапазоне ключей быстрее всего работал CountingSort, а QuickSort заметно замедлился на данных с большим количеством повторов.

6. Задание 6

Составили сводную таблицу свойств шести алгоритмов сортировки. Сравнили время работы в лучшем, среднем и худшем случаях, дополнительную память, устойчивость и ограничения применения. В таблице n - количество элементов массива, k = max − min - ширина диапазона ключей, d - количество десятичных разрядов максимального ключа. CountingSort использует k + 1 счётчиков. Устойчивость означает сохранение исходного порядка элементов с одинаковыми ключами, а дополнительная память - память сверх исходного массива. При оценке учли особенности наших реализаций: QuickSort создаёт копию списка, а HeapSort сортирует исходный список на месте.

Алгоритм	**Лучшее время**	**Среднее время**	**Худшее время**	**Дополнительная память**	**Устойчивость**	**Ограничения**
MergeSort	O(n log n)	O(n log n)	O(n log n)	O(n)	Да	Сравнимые ключи
CountingSort	O(n + k)	O(n + k)	O(n + k)	O(n + k)	Да	Целые ключи от 0 до key_max; широкий диапазон требует много памяти
RadixSort LSD	O(d(n + 10))	O(d(n + 10))	O(d(n + 10))	O(n + 10)	Да	Неотрицательные целые ключи; устойчивый внутренний проход
HeapSort	O(n)*	O(n log n)	O(n log n)	O(1)	Нет	Сравнимые элементы
QuickSort из ЛР3	O(n log n)	O(n log n)**	$O(n^2)$	O(n)	Нет	Большое число одинаковых ключей ухудшает разделение
sorted()	O(n)	O(n log n)	O(n log n)	O(n)	Да	Сравнимые ключи

Таблица 1 - Свойства алгоритмов сортировки

Вывод: Выбор сортировки зависит от диапазона ключей, доступной памяти и необходимости сохранять порядок равных элементов. CountingSort подходит для узкого

диапазона целых ключей, а HeapSort позволяет сортировать на месте с постоянной дополнительной памятью.

7. Задание 7

Проверили каждую сортировку на граничных случаях и 400 случайных массивах. Контролировали порядок элементов, сохранение всех исходных значений и совпадение результата с sorted(). Устойчивость MergeSort, CountingSort и RadixSort проверили на парах «ключ+порядковый номер». Дополнительно проверили устойчивость поразрядного прохода и свойство построенной кучи: родитель должен быть не меньше своих потомков.

```
def check_sort(fn, signed: bool = False) -> None:  1 usage  Lena *
    """Граничные случаи, порядок, сохранность элементов и сверка с sorted()."""
    cases = [
        [], [7], [0], [2, 1], [1, 2], [5, 5, 5],
        [1, 2, 3, 4, 5], [5, 4, 3, 2, 1],
    ]
    if signed:
        cases.append([-3, 0, -1, -3, 2])
    for seed in (0, 46):
        rng = random.Random(seed)
        for _ in range(200):
            cases.append([
                rng.randint( a: 0,  b: 999) for _ in range(rng.randint( a: 0,  b: 60))
            ])
    for a in cases:
        result = fn(list(a))
        expect(isinstance(result, list),  message: "Результат должен быть списком")
        expect(all(result[i] <= result[i + 1]
                   for i in range(len(result) - 1)),
               message: f"Нарушен порядок на входе {a}")
        expected = sorted(a)
        expect(sorted(result) == expected,  message: f"Изменён набор элементов на входе {a}")
        expect(result == expected,  message: f"Расхождение с sorted() на входе {a}")


def check_stability(fn) -> None:  3 usages  new *
    """Порядок номеров при одинаковых ключах должен сохраняться."""
    rng = random.Random(46)
    pairs = [(rng.randint( a: 0,  b: 99), i) for i in range(200)]
    expect(fn(list(pairs)) == sorted(pairs, key=lambda p: p[0]),
           message: "Нарушена устойчивость на парах (ключ, порядковый номер)")
```

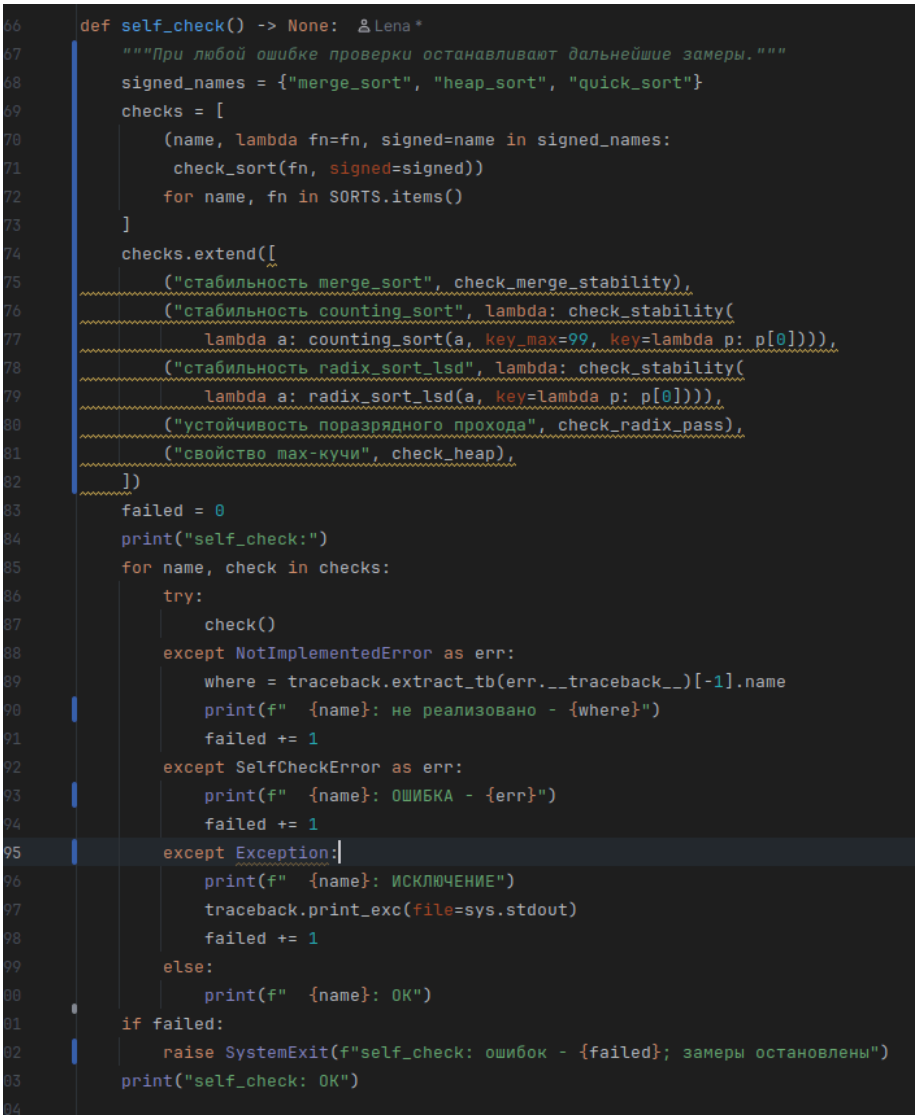
Рисунок 21 - Проверка правильности и устойчивости сортировок

```python
def check_merge_stability() -> None:  1 usage  Lena *
    """Сохраняем проверку заготовки и добавляем проверку с seed=46."""
    rng = random.Random(0)
    pairs = [(rng.randint( a: 0,  b: 5), i) for i in range(100)]
    expect(merge_sort(pairs, key=lambda p: p[0])
           == sorted(pairs, key=lambda p: p[0]),
           message: "merge_sort нарушает порядок равных ключей")
    check_stability(lambda a: merge_sort(a, key=lambda p: p[0]))


def check_radix_pass() -> None:  1 usage  new *
    """Проверка цифры и устойчивости отдельного поразрядного прохода."""
    pairs = [(21, 0), (11, 1), (22, 2), (12, 3), (21, 4), (11, 5)]
    for exp in (1, 10, 100):
        result = counting_sort_digit(pairs, exp, key=lambda p: p[0])
        expected = sorted(pairs, key=lambda p: (p[0] // exp) % 10)
        expect(result == expected,  message: f"Неверный или неустойчивый проход exp={exp}")


def check_heap() -> None:  1 usage  new *
    """После построения кучи родитель не меньше каждого своего потомка."""
    rng = random.Random(46)
    for n in (0, 1, 2, 10, 100):
        data = [rng.randint( a: 0,  b: 999) for _ in range(n)]
        expected = sorted(data)
        build_heap(data)
        expect(all(data[(i - 1) // 2] >= data[i] for i in range(1, n)),
               message: "Нарушено свойство max-кучи")
        expect(sorted(data) == expected,  message: "Построение кучи изменило набор элементов")
```

Рисунок 22 - Проверки устойчивости MergeSort, поразрядного прохода и свойства кучи



```
def self_check() -> None:  Lena *
    """При любой ошибке проверки останавливают дальнейшие замеры."""
    signed_names = {"merge_sort", "heap_sort", "quick_sort"}
    checks = [
        (name, lambda fn=fn, signed=name in signed_names:
         check_sort(fn, signed=signed))
        for name, fn in SORTS.items()
    ]
    checks.extend([
        ("стабильность merge_sort", check_merge_stability),
        ("стабильность counting_sort", lambda: check_stability(
            lambda a: counting_sort(a, key_max=99, key=lambda p: p[0]))),
        ("стабильность radix_sort_lsd", lambda: check_stability(
            lambda a: radix_sort_lsd(a, key=lambda p: p[0]))),
        ("устойчивость поразрядного прохода", check_radix_pass),
        ("свойство max-кучи", check_heap),
    ])
    failed = 0
    print("self_check:")
    for name, check in checks:
        try:
            check()
        except NotImplementedError as err:
            where = traceback.extract_tb(err.__traceback__)[-1].name
            print(f"  {name}: не реализовано - {where}")
            failed += 1
        except SelfCheckError as err:
            print(f"  {name}: ОШИБКА - {err}")
            failed += 1
        except Exception:
            print(f"  {name}: ИСКЛЮЧЕНИЕ")
            traceback.print_exc(file=sys.stdout)
            failed += 1
        else:
            print(f"  {name}: OK")
    if failed:
        raise SystemExit(f"self_check: ошибок - {failed}; замеры остановлены")
    print("self_check: OK")
```

Рисунок 23 - Общий запуск проверок и обработка ошибок

```
  merge_sort: OK
  counting_sort: OK
  radix_sort_lsd: OK
  heap_sort: OK
  quick_sort: OK
  стабильность merge_sort: OK
  стабильность counting_sort: OK
  стабильность radix_sort_lsd: OK
  устойчивость поразрядного прохода: OK
  свойство max-кучи: OK
self_check: OK
```

Рисунок 24 - Результаты проверки алгоритмов сортировки

Вывод: Все проверки прошли успешно, результаты совпали с sorted(). Подтверждены устойчивость заявленных сортировок и поразрядного прохода, а также правильность построения кучи.

Заключение

Реализованы четыре сортировки и сравнены их время работы, память, устойчивость и ограничения применения. На данных варианта 16 быстрее всего работал sorted(), а среди собственных реализаций на узком диапазоне ключей CountingSort. Выбор алгоритма зависит от характера данных, доступной памяти и необходимости сохранять порядок равных ключей.